

Generative Adversarial Networks:

Specific case of general f -divergence class of func^s we saw

* Choice for f -divergence:

$$f(u) = u \log u - (u+1) \log (u+1) \quad \text{Similar to JSD}$$

$$f^*(t) = -\log (1 - \exp(t)) \quad \text{Conjugate to } f(u)$$

$$\text{dom } f^* = \mathbb{R}^-$$

choose σ_f so it projects to $\mathbb{R}^-$

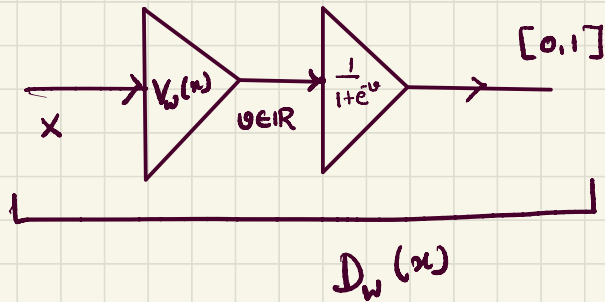
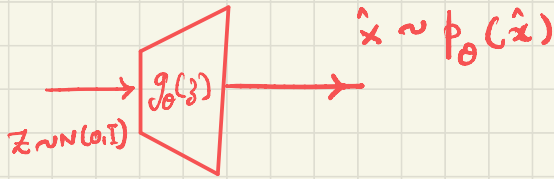
$$\sigma_f(v) = -\log (1 + e^{-v})$$

$$J(\theta, w) = \mathbb{E}_{p_x} \left[\sigma_f(v_w(x)) \right] - \mathbb{E}_{p_\theta} \left[f^*(\sigma_f(v_w(x))) \right]$$

$$J(\theta, w) = \mathbb{E}_{p_x} \left[\log D_w(x) \right] + \mathbb{E}_{p_\theta} \left[\log (1 - D_w(x)) \right]$$

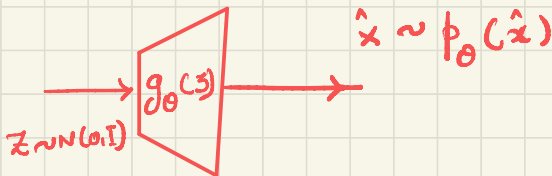
where $D_w(x) = \frac{1}{1 + e^{-v_w(x)}}$: Sigmoid funcⁿ

Implementation.

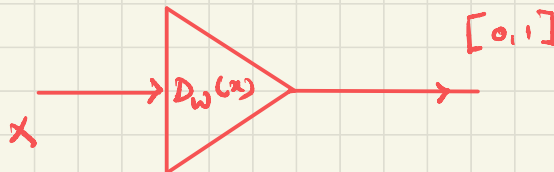


GAN Architecture:

Generator



Discriminator / Binary
classification



Loss func

$$J_{\text{GAN}}(\theta, w) = \mathbb{E}_{x \sim p_x} [\log D_w(x)] + \mathbb{E}_{\hat{x} \sim p_\theta} [\log (1 - D_w(\hat{x}))]$$

Implementation of GAN in practice.

Input : $D = \{x_1, x_2, \dots, x_n\} \sim \text{iid } p_x$

Choose one for generator & Discriminator w/o

$$w^* = \arg \max_w \mathbb{E}_{p_x} (\log D_w(x)) + \mathbb{E}_{p_\theta} (\log (1 - D_w(x)))$$

now approximating $\mathbb{E}$ with Sample average & using Batches.

$$w^* = \arg \max_w \left[\frac{1}{B_1} \sum_{i=1}^{B_1} \log D_w(x_i) + \frac{1}{B_2} \sum_{j=1}^{B_2} \log (1 - D_w(\hat{x}_j)) \right]$$

$$x_1, x_2, \dots, x_{B_1} \sim p_x$$

$$\hat{x}_1, \hat{x}_2, \dots, \hat{x}_{B_2} \sim p_\theta$$

$$w^{t+1} \leftarrow w^t + \alpha_1 \nabla_w J_{GAN}(\theta, w)$$

(Gradient Ascent)

One Gradient step
through Discriminator

While we are training Discriminator w , parameters of generator θ are constant

$$\theta^* = \underset{\theta}{\operatorname{argmin}} J_{GAN}(\theta, w)$$

$$= \underset{\theta}{\operatorname{argmin}} \left[\frac{1}{B_1} \sum_{i=1}^{B_1} \log D_w(x_i) + \frac{1}{B_2} \sum_{j=1}^{B_2} \log (1 - D_w(\hat{x}_j)) \right]$$

independent of θ

$$x_1, x_2, \dots, x_{B_1} \sim p_x$$

$$\hat{x}_1, \hat{x}_2, \dots, \hat{x}_{B_2} \sim p_o$$

$$\hat{x}_j = g_{\theta}(z_j)$$

$$= \underset{\theta}{\operatorname{argmin}} \left[\frac{1}{B_2} \sum_{j=1}^{B_2} \log (1 - D_w (g_\theta(\bar{z}_j))) \right]$$

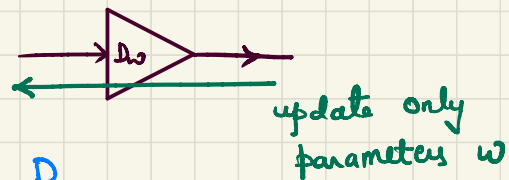
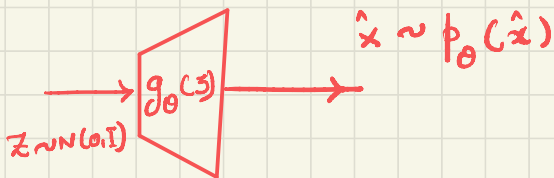
$$\theta^{t+1} \leftarrow \theta^t - \alpha_2 \nabla_{\theta} J_{\text{GAN}}(\theta, w)$$

One grad step through Generator.

* In this step parameters of w are kept as Const.

To train the Discriminator:

keep ' θ ' Const



$$B_1 = \{x_1, x_2, \dots, x_{B_1}\} \subset D$$

$$J_{\text{GAN}}(\theta, w) = \left[\frac{1}{B_1} \sum_{i=1}^{B_1} \log D_w(x_i) + \frac{1}{B_2} \sum_{j=1}^{B_2} \log (1 - D_w(g_\theta(\bar{z}_j))) \right]$$

Sample

$$z_1, \dots, z_{B_2} \sim \mathcal{N}(0, I)$$

pass through $g_0(z)$

now get

$$g_0(z_1), \dots, g_0(z_{B_2})$$

pass these through $D_0(\cdot)$ to get the 2nd term

Now get the gradient $\nabla_w J_{GAN}(\theta, w)$

now update the weights of Discriminator.

To Train the Generator network:

keep the Discriminator parameters (w) fixed.

→ Sample $z_1, \dots, z_{B_2} \sim \mathcal{N}(0, I)$

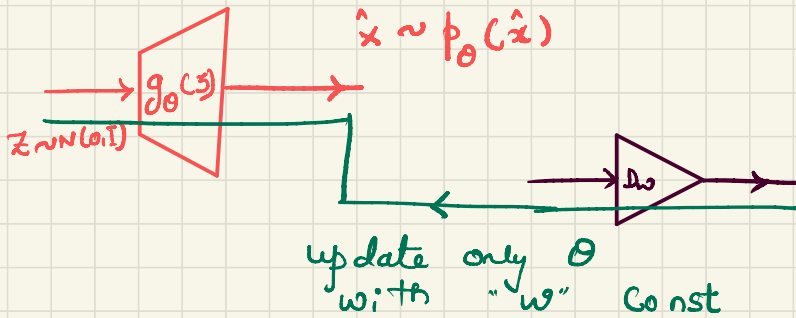
get $g_0(z_1), \dots, g_0(z_{B_2})$

→ pass these through Discriminator.

then Compute $D_w(g_0(z_j))$

$$\frac{1}{B_2} \sum_{j=1}^{B_2} \log(1 - D_w(g_\theta(z_j)))$$

now compute the gradient $\nabla_{\theta} J_{GAN}(\theta, w)$



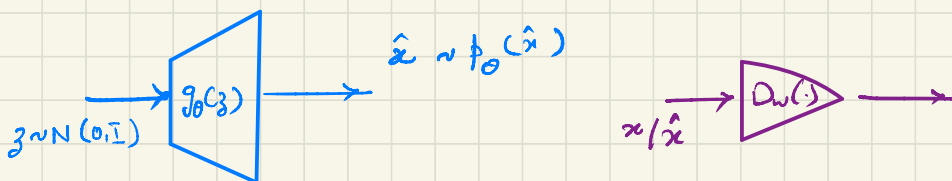
- One step through Disc, One step through Gen
- One step through Disc, 5 steps through Gen
- 5 step through Disc, 1 step through Gen

Look @ the quality of generated images
 & stop once we reach certain quality.

Lecture 4:

Interpretation of a GAN as Classifier Guided Generative Sampling

$$D = \{x_1, x_2, \dots, x_n\} \sim \text{iid } p_x$$



Goal: $p_\theta(\hat{x})$ to be close to $p_x(x)$

Suppose there is a binary classifier

$$D_w(x) = \begin{cases} 1 & \text{if } x \sim p_x \\ 0 & \text{if } x \sim p_\theta \end{cases}$$

$D_w(x)$ is a binary classifier b/w the samples of p_x & p_θ

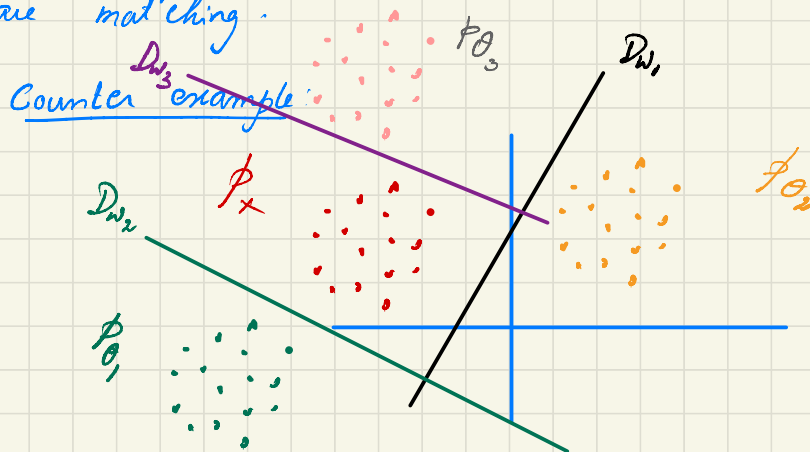
question: Can $D_w(x)$ be used for making p_θ & p_x

close?

Answer: Tweak θ (parameters of g_θ) till the classifier fails to distinguish the samples b/w p_x & p_θ

This will happen when p_θ is similar to p_x

however failure of the classifier doesn't imply that distributions are matching.



change of parameter move the cluster

In this case D_{w1} fails as it's not able to distinguish b/w p_0 & p_x

→ matching means p_x & p_0 should overlap, then the failure of classifier makes sense.

Soln: Hence the classifier has to be simultaneously tweaked along with the generator.

* There is a possibility that p_0 can toggle b/w p_{01} & p_{02} alternatively to fool the discriminator

Formulation of Classifier Guided Sampler:

Suppose $D_w: X \rightarrow [0, 1]$

Let $D_w(x)$ represent the likelihood of sample x coming from p_x .

The objective is to maximize the log likelihood of 'x' coming from p_x

$$w^* = \arg \max_w \left[\mathbb{E}_{p_x} \log D_w(x) \right] : \text{maximize the likelihood of } x \sim p_x$$

we also want the classifier should also maximize the likelihood of $\hat{x}$ not coming from p_x

when $\hat{x} \sim p_0$

$$1 - D_w(\hat{x})$$

likelihood of $\hat{x}$ not from p_x

$$\arg \max_w \mathbb{E}_{p_0} \left[\log (1 - D_w(\hat{x})) \right] = w^*$$

maximizing the likelihood of $\hat{x}$ not from p_x
 $\hat{x} \sim p_0$

The combined objective for classifier training,

$$w^* = \underset{w}{\operatorname{argmax}} \mathbb{E}_{p_x} [\log D_w(x)] + \mathbb{E}_{p_0} [\log (1 - D_w(\hat{x}))]$$

This is the lower bound that we saw earlier.

Now the objective for generator $g_\theta(z)$ is such that the classifier has to fail.

Invent the optimization for the classifier.

$$\theta^* = \underset{\theta}{\operatorname{argmin}} J(\theta, w)$$

$$\theta^*, w^* = \underset{\theta}{\operatorname{argmin}} \underset{w}{\operatorname{argmax}} \left[\mathbb{E}_{p_x} \log D_w(x) + \mathbb{E}_{p_0} \log (1 - D_w(\hat{x})) \right]$$

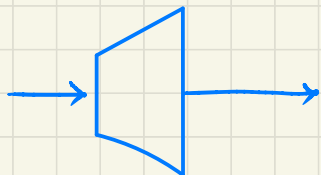
$$\theta^*, w^* = \underset{\theta}{\operatorname{argmin}} \underset{w}{\operatorname{max}} [J(\theta, w)]$$

adversarial part.

DC GAN: Deep Convolutional GAN.

Typically in a GAN

$$z \in \mathbb{R}^k, \quad x \in \mathbb{R}^d \quad k \ll d$$



increases : MLP / FC

Typically Reshape to size of image.

In DC-GAN, up convolutional (a) Transposed Conv layers are used in generators.

Increase the size of z using up conv., till it reaches the size of data

$$x \in \mathbb{R}^{8 \times 8 \times 3}$$

* Reshape is not needed.

Conditional GAN (c) C-GAN:

In a Vanilla GAN, There is no Control over the Data generated.

Generate to a Particular class.

$$D = \left\{ (x_i, y_i) \right\}_{i=1}^n \sim p_{xy}$$

Data Similar to discriminative tasks.

x : Images

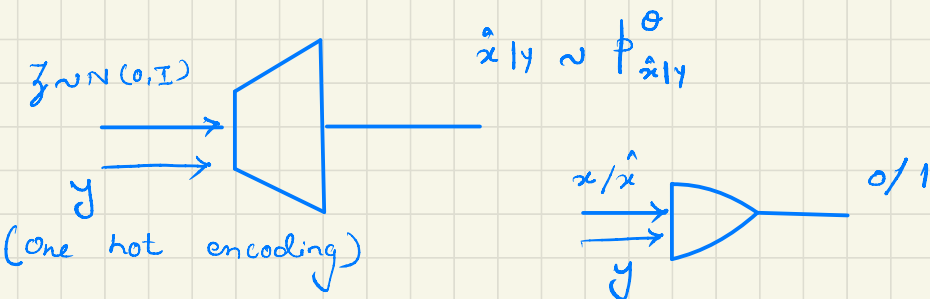
y : Label / Textual embeddings.

objective is to sample from CondⁿL distribution $p_{x|y}$ instead of p_x

Sample from CondⁿL not marginal

Solution: Estimate $p_{x|y}$ & make p_θ approach $p_{x|y}$ for all y .

Modify the generator & discriminator to operate on CondⁿL distribution



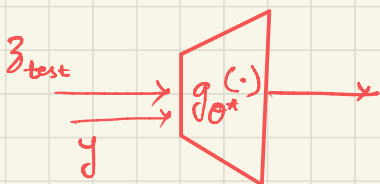
$$J(\theta, \omega) = \mathbb{E}_{x, y \sim p_{x|y}} \log D_\omega(x, y) +$$

$$\mathbb{E}_{\hat{x}, y \sim p_{\hat{x}|y}^\theta} \log (1 - D_\omega(\hat{x}, y))$$

Inference with GANs:

Input: A trained generator n/w g_{θ^*}

Output: A sample from p_x which is not in D



$\hat{x}_{\text{test}}$ corresponding to class y .

Lecture 05:

Wasserstein's GAN:

- * The saddle point problem, diff to obtain optimal. (of GAN)
- * f -divergence minimization leads to unstable training.
- * The goodness depends on how tight lower bound we are getting.

Question: what makes GAN training unstable?

Manifold hypothesis: Distributions of real data

(Such as images) lie over a lower dim manifold in the ambient space. ($\mathbb{R}^d$)